

# Unit 17

## Reporting & Professional Communication + Capstone Kickoff

Module 5 — Putting It Together · Week 17

The client isn't buying the hack. They're buying **the report**.

# Where we are

---

- For 16 weeks you learned **how** to find and demonstrate weaknesses.
- Now you learn the part that **pays the bills**: the report.
- A brilliant test with a sloppy report is a **failed engagement**.
- Then we **kick off the capstone** — the project you finish and report on next week.

Clients pay for the document that tells them what's wrong, how bad it is, and how to fix it.

# What you'll be able to do

---

By the end of this unit you can:

- Explain **why the report is the real deliverable** of a pentest.
- Identify the standard report sections: **exec summary, methodology, findings, remediation, appendices**.
- Write the same issue for **two audiences**: executives and technical staff.
- Assign a **severity** (Low/Med/High/Critical) and justify it with **likelihood × impact**.
- Document a finding with **evidence, impact, and specific remediation**.

# What you'll be able to do (cont.)

---

- Apply the **penetration-test report rubric** to evaluate and improve a section.
- Explain the **ethics of accurate reporting** — no exaggerating, no minimizing, no hiding.
- Produce a **complete sample report** from a prior lab.
- Receive and confirm your **capstone target assignment**.

# Vocabulary — the document

| Term                    | Meaning                                                              |
|-------------------------|----------------------------------------------------------------------|
| Deliverable             | What the client actually receives and pays for — here, the report.   |
| Penetration-test report | Formal doc: what was tested, found, how serious, how to fix.         |
| Executive summary       | Short, plain-language overview for busy, non-technical leaders.      |
| Methodology             | The section explaining <i>how</i> the test was done.                 |
| Appendix                | Supporting material at the end (raw output, host lists, references). |

# Vocabulary — the finding

| Term                   | Meaning                                                              |
|------------------------|----------------------------------------------------------------------|
| Finding                | A single security issue, written up with evidence, severity, impact. |
| Evidence               | Proof a finding is real — labeled screenshots, commands, hosts/URLs. |
| Severity / risk rating | How serious it is — Low / Medium / High / Critical.                  |
| Impact                 | What could actually go wrong for the organization.                   |
| Likelihood             | How easy or probable it is that an attacker exploits it.             |
| Remediation            | The specific, actionable fix recommended.                            |

# Vocabulary — risk & communication

| Term           | Meaning                                                                |
|----------------|------------------------------------------------------------------------|
| Recommendation | Broader advice to improve security beyond one fix.                     |
| CVSS           | Common Vulnerability Scoring System — turns factors into a 0-10 score. |
| Audience       | Who you're writing for — executives vs. technical staff differ.        |
| Scope          | The agreed boundary of what was authorized — restated in the report.   |
| Accuracy       | Reporting truthfully — not exaggerated, not minimized, not hidden.     |
| Capstone       | The end-of-course project: complete a CTF target, report, present.     |

# Ethics & Authorization

This week's thread is **accuracy**. Two failures are equally serious.



# The two ways to betray trust

---

- **Exaggerating** a finding to look impressive — scaring a client into wasting money, or crying wolf.
- **Minimizing or hiding** a finding because it's awkward, you ran out of time, or you like the client.
- Both **distort the truth the client paid for**.

A professional reports what is true — with evidence — and rates it honestly.

# Scope lives in the report too

---

- The report **restates the scope and authorization** of the test.
- It is the **written record** proving the test was permitted and stayed in bounds.
- This is both professional practice and the ethics thread of the whole course.

The report is the document a client makes real decisions and spends real money on.

# Day 1

Why the Report Is the Real Product + Report Anatomy

# Warm-up

---

A tester spends two weeks brilliantly breaking into a client's network, then hands over a **one-paragraph email**.

Did the client get what they paid for? Why not?

# The big idea

---

- Clients **pay for the report, not the hack**.
- A finding that isn't clearly written and reproducible **effectively doesn't exist** for the client.
- In industry, the report **is** the job.

"The hacking was the fun part" — true, but the report is the product.

# Anatomy of a pentest report

| Section           | Who reads it        | What it answers                 |
|-------------------|---------------------|---------------------------------|
| Executive summary | Leaders             | How exposed are we? What first? |
| Methodology       | Technical reviewers | How did you test?               |
| Findings          | IT / fixers         | What's wrong, with proof        |
| Remediation       | IT / leaders        | How do we fix it?               |
| Appendices        | Technical staff     | Raw output, references          |

# Guided practice (Day 1)

---

Sort each scrambled item into the right section:

- A labeled screenshot → **Findings (evidence)**
- A CEO-friendly paragraph → **Executive summary**
- Raw `nmap` output → **Appendices**
- A fix recommendation → **Remediation**
- A risk rating → **Findings (severity)**

Then begin **Lab Part A**: pick a prior-unit finding and fill in the basic facts.

**Exit ticket:** Name the five sections; which one does a non-technical CEO read first?

# Day 2

## Findings, Evidence, Severity & CVSS Awareness



# Warm-up

---

Two versions of the same finding:

- A) A labeled screenshot + exact steps to reproduce.
- B) "The site is hackable."

Which can a developer actually **act on**?

# Anatomy of a single finding

---

Every finding needs:

- **Title** — clear and specific (not "the website is bad").
- **Affected asset** — host, IP, or URL.
- **Evidence** — labeled screenshots, exact commands, URLs.
- **Severity** — Low / Med / High / Critical.
- **Impact** — what an attacker could actually do.
- **Remediation** — the specific fix.

If a reader can't reproduce it from your write-up, it isn't done.

# Labeling evidence

Figure 1: login form accepts ' OR 1=1 -- (authentication bypass)

- Every screenshot gets a **figure number and caption**.
- Commands are written **exactly** so a reader can repeat them.
- Re-teach trigger: "trust me, it worked" with no evidence.

Good reports come from good lab journals. You can only report what you wrote down.

# Severity = likelihood × impact

| Rating          | Rough meaning                                 | Example                                                    |
|-----------------|-----------------------------------------------|------------------------------------------------------------|
| <b>Critical</b> | Easy AND severe (full system / customer data) | Unauth RCE; SQLi dumping the user table                    |
| <b>High</b>     | Serious; takes some effort or a condition     | Stored XSS stealing admin sessions; default admin password |
| <b>Medium</b>   | Real risk, limited or harder to exploit       | Reflected XSS needing a click; directory listing           |
| <b>Low</b>      | Minor; little direct impact                   | Verbose error leaking version; missing header              |

# CVSS — awareness only

---

- **CVSS** = an industry method that turns factors into a **0-10 score**.
- It considers things like **how easy** the attack is and **how much damage** it does.
- You should know **it exists** and **roughly what feeds it** — not compute vectors by hand.

The online calculator is an extension task, not a requirement.

# Guided + independent practice (Day 2)

---

In pairs, rate and **justify** three findings:

- Default admin password
- Reflected XSS
- Missing security header

Then **Lab Part B:** write your finding(s) with evidence and a justified severity.

**Exit ticket:** What three things must every finding include? Rate one example and justify it in a sentence.

# Day 3

## Writing for Two Audiences

# Warm-up

---

Explain SQL injection to (a) your **principal** and (b) a **database administrator**.

How does the language change?



# The two-audience problem

|          | Executive summary             | Technical finding                |
|----------|-------------------------------|----------------------------------|
| Reader   | Busy, non-technical leader    | Staff who will fix it            |
| Language | Plain, risk-focused           | Precise, reproducible            |
| Contains | "How exposed? What to do?"    | Exact param, payload, URL, steps |
| Avoids   | Commands, un-glossed acronyms | Vagueness                        |

Same truth, different **altitude**.

# Same finding, two ways

---

## Technical finding:

Unsanitized input in the login form permits UNION-based SQL injection on `/login`.

## Executive summary version:

An attacker could currently access customer records through the login page. We recommend prioritizing a fix this quarter.

# Rules for the executive summary

---

- **No commands.** No `sqlmap`, no payloads, no code.
- **No un-glossed acronyms.** Spell it out or explain it in plain words.
- 2–4 sentences answering: *how exposed are we, and what should we do?*
- A good test: could the **principal** read it and know how worried to be and what to prioritize?

This is the hardest skill. Jargon leaks. Have peers hunt for it.

# Guided + independent practice (Day 3)

---

- Take your Day-2 technical finding; draft a **2-3 sentence executive-summary version** — no jargon.
- Peer-swap and **flag any jargon** that slipped through.
- Then **Lab Part C:** write the executive summary + the remediation/recommendations.

**Exit ticket:** Rewrite for an executive: *"Unsanitized input in the login form permits UNION-based SQL injection."*

# Remediation must be specific

| Vague (not OK)        | Specific (OK)                                                      |
|-----------------------|--------------------------------------------------------------------|
| "Make it more secure" | "Use parameterized queries / prepared statements for all DB calls" |
| "Fix the login"       | "Validate input server-side; reject unexpected characters"         |
| "Patch it"            | "Change the default admin password; enforce a password policy"     |

Remediation a developer can act on **today**.

# Day 4

Assemble the Full Report + Peer Review

# Warm-up

---

Read the **report rubric** aloud.

Which row do you think you'll **lose the most points** on, and how will you fix that today?

# Professional polish

---

- **Restate scope/authorization** at the top.
- **Label every screenshot** (Figure 1, 2, 3...).
- **Number your findings.**
- Use **consistent terminology** throughout.
- Add an **appendix** for raw tool output.

Reports come from notes — thin lab journals bite you here.



# The report template

```
# Penetration Test Report – [Target / Lab Name]
**Tester:** ... **Date:** ...
**Scope & authorization:** authorized test of [target], Unit __, with permission.

## 1. Executive summary      (plain language, no jargon)
## 2. Methodology            (recon → scan → exploit → document)
## 3. Findings               (title, asset, severity+justification, evidence, impact)
## 4. Remediation & recommendations
## 5. Appendices             (raw output, references)
```

# Assemble + peer review (Day 4)

---

- Assemble Parts A–C into one **complete sample report** (all five sections + appendix).
- **Peer review (15 min):** swap with a partner; score against the **report rubric**.
- Give one **glow** (strength) and one **grow** (improvement).

**Exit ticket:** Hand in the rubric you used to score your partner, with your glow + grow.

# The report rubric

| Criteria          | Exemplary (4)                       | Beginning (1) |
|-------------------|-------------------------------------|---------------|
| Executive summary | Clear, non-technical, accurate risk | Missing       |
| Methodology       | Phases described & justified        | Missing       |
| Findings          | Each has evidence, severity, impact | Missing       |
| Remediation       | Specific, actionable                | Missing       |
| Communication     | Polished, organized, correct terms  | Unclear       |

# Day 5

Revise & Submit + Capstone Kickoff

# Warm-up

---

What's **one change** you're making to your report based on yesterday's peer feedback?

# Capstone Kickoff

---

Next week, each **team** will:

1. Run the **full attack lifecycle** on an authorized target:  
recon → scan/enumerate → exploit → privilege-escalate → document.
2. Write a **full report** (report rubric).
3. Give a **short presentation** (presentation rubric).

The dress rehearsal you just did is the real thing next week.

# Capstone rules

---

- Targets are **picoCTF** and/or an **approved beginner TryHackMe room** — ONLY.
- **No real, third-party, or out-of-scope systems. Ever.**
- Teams of 2–3; everyone documents.
- You'll get: team assignment, target list, checklist, due dates.

Restate the safety rule: the approved list is the **entire** universe of legal targets.

# Team task (Day 5)

---

- Finish revising and **submit the sample report**.
- With your team, **confirm your target** and write your **capstone scope statement**:

*"We are authorized to test only [the approved target our instructor assigned]. No other systems are in scope."*

**Exit ticket:** Submit team name, target, and your one-sentence scope/authorization statement.



# Unit recap

---

- Clients pay for the **report**, not the hack.
- Five sections: **exec summary, methodology, findings, remediation, appendices**.
- Every finding: **evidence + severity + impact**, with **specific remediation**.
- Severity = **likelihood × impact** (no inflation).
- Same truth, **two audiences**, different altitude.
- **Accuracy** is an ethical duty — no exaggerating, no hiding.

# Exit Discussion

You found a **Critical** flaw — but it was *embarrassingly* easy (a default password).

You worry the client will think the test was "too simple."

Are you tempted to **dress it up** or **downplay it**? What does honest reporting require, and why does the client's safety outrank your ego?